Задача 4. Инфиксные арифметические выражения

4a. Вычисление выражения через eval

Дано арифметическое выражение в инфиксной форме, представленное списком. Элементы чередуются: операнд, оператор, операнд и т. д. Операндом может быть число или символ (имя переменной).

```
'(x + 4 * y - 5 / 2)
```

Допустимые бинарные операторы: +, -, *, /. Значения переменных передаются отдельно в словаре bindings, где ключи — символы, а значения — числа.

Реализуйте функцию:

```
(eval-expression expr bindings)
```

Функция должна:

1. Преобразовать инфиксное выражение в корректную префиксную форму Clojure (S-выражение).

2. Учитывать приоритет операций: * и / имеют более высокий приоритет, чем + и -. Операции одного приоритета группировать слева направо.

3. Обеспечить разрешение всех переменных по словарю bindings. При отсутствии значения переменной сообщать об ошибке.

4. Вычислить построенную форму с помощью eval и вернуть результат.

Примеры:

```
(eval-expression '(1 + 4 * 3 - 5 / 2) {})
;; => 21/2

(eval-expression '(x + 4 * y - 5 / 2) '{x 10, y 4})
;; => 47/2

(eval-expression '(x - y - 2) '{x 10, y 3})
;; => 5

(eval-expression '(x / y * 2) '{x 8, y 4})
;; => 4
```

Например, для второго вызова итоговая форма должна быть эквивалентна:

```
(let [x 10, y 4]
  (- (+ x (* 4 y)) (/ 5 2)))
```

Ограничение. Операции должны вычисляться через eval, а не вручную через case, cond или аналогичный диспетчер.

4b. Инфиксный синтаксис с помощью макроса

Реализуйте макрос infix, позволяющий записывать арифметические выражения в инфиксной форме непосредственно в программе:

```
(defmacro infix [expr]
  ...)

(let [x 10 y 4]
  (infix (x + 4 * y - 5 / 2)))
;; => 47/2
```

Макрос должен:

1. Принимать исходную форму выражения без предварительного вычисления её элементов и преобразовывать её с учётом приоритета и левосторонней ассоциативности операций.
2. Возвращать корректную префиксную форму, которая будет вычислена обычным механизмом Clojure.
3. Работать с локальными переменными из лексического окружения вызывающего кода — без передачи отдельного словаря bindings.
4. Не вызывать eval внутри макроса.

Примеры:

```
(infix (2 + 3 * 4))
;; => 14

(infix (10 - 3 - 2))
;; => 5

(let [x 8 y 4]
  (infix (x / y * 2)))
;; => 4
```

Для выражения (x + 4 * y - 5 / 2) результат macroexpand-1 должен быть эквивалентен форме:

```
(- (+ x (* 4 y))
   (/ 5 2))
```

Общее требование к 4a и 4b. Логику преобразования инфиксного выражения в префиксную форму вынесите в одну обычную функцию и используйте в обеих реализациях. Не дублируйте алгоритм разбора в макросе.